Projects

Project Finding Phase

Please follow this order ☺

1. Find a teammate (chat after the lecture, ask on Slack)
2. Find a project together
3. Get in touch with the corresponding TA (E-Mail or Slack)
4. After confirmation, add your team & topic to the public group list (will be shared in Moodle), so that others see which projects are taken.

Text

Where & How to find a project

- Pick one presented direction that interests you the most and discuss with the corresponding TA.

- If those topics are not interesting to you, check out our publications and chat with Matthias/TAs: http://www.niessnerlab.org/publications.html

- It's highly encouraged to think about your own project idea!

General Guidelines

- The presentations and final report really matter - Make sure you reach out to your TA for help with technical questions, presentations, report, etc.
- Push your project from the beginning rather than at the end -> This will spare you more time for your final report and exam.
- Schedule regular meetings with your TA.
- Send regular updates about your project.

Transforming and Optimizing 3D Scene Representations

Nicolas von Lützow
nicolas.von-luetzow@tum.de

Project Directions

INTRO: Novel view synthesis using Neural Radiance Fields (NeRF, Mildenhall et al.)

n input images

Optimize NeRF scene representation

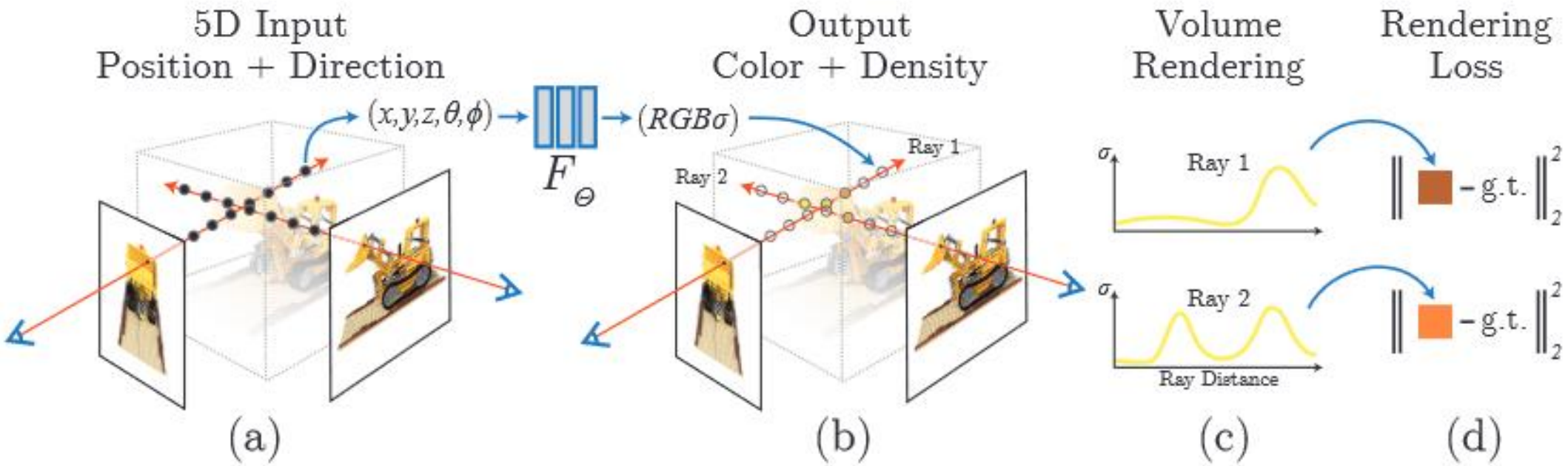
Render novel views

Input: Set of images of a scene with intrinsic & extrinsic camera parameters

Output: Novel views from arbitrary camera poses

Project Directions

INTRO: Novel view synthesis using Neural Radiance Fields (NeRF, Mildenhall et al.)

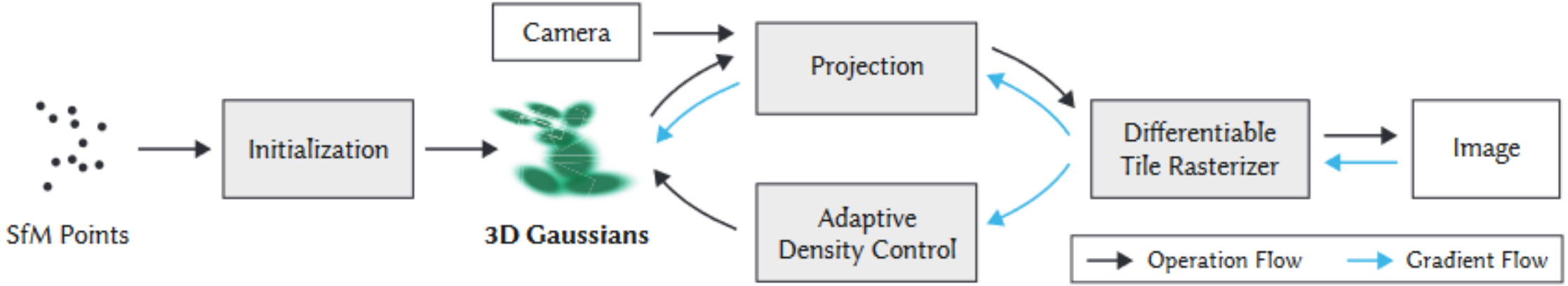


Input: Set of images of a scene with intrinsic & extrinsic camera parameters

Output: Novel views from arbitrary camera poses

Project Directions

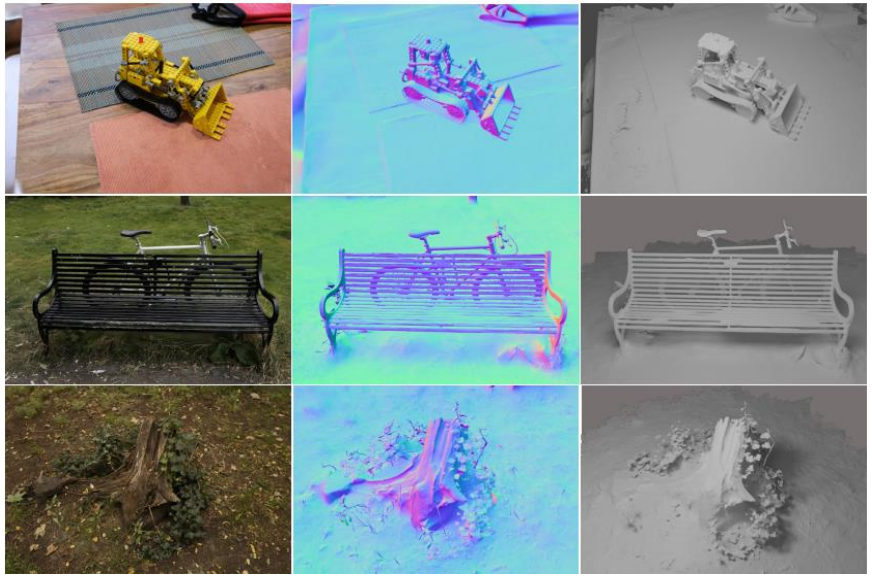
INTRO: Novel view synthesis using 3D Gaussians (3DGS, Kerbl et al.)



- Very high quality
- Fast optimization
- Super fast rendering

Project Directions

Feed-Forward Radiance Field Conversion



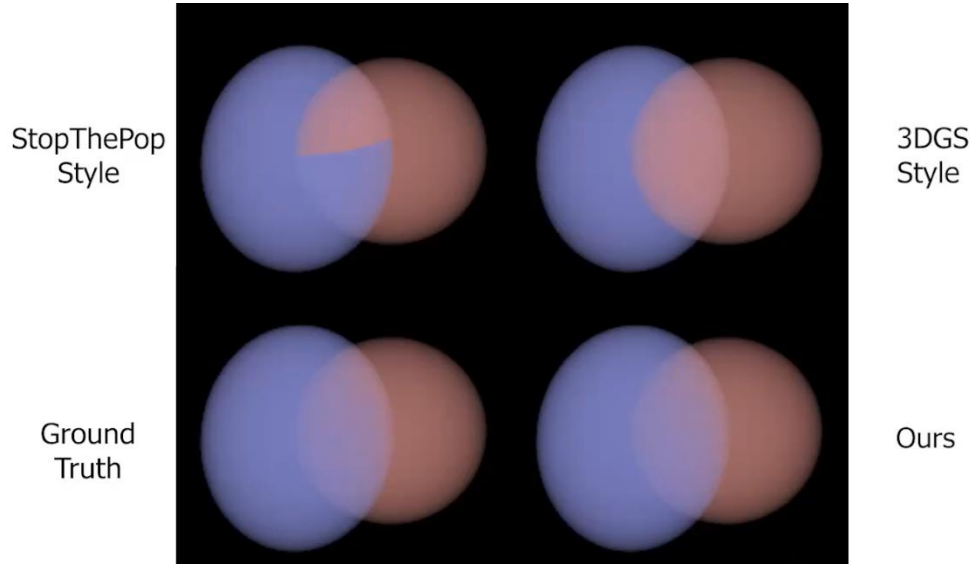
- **Goal:** Learn to translate between different radiance field formats
- **Setup:**
  - **Input**: Optimized 3D Gaussian Scene
  - **Output**: Equivalent 2D Gaussian Scene
- **Method:**
  - Design an autoencoder mapping 3D <-> 2D Gaussian scenes
  - Train for appearance (and geometric) consistency
- **Questions:**
  - Can we improve depth estimates and normals of the 3DGS scenes?
  - Can we extend this to other 3D representations (3DCS, LinPrim, meshes)?
    - Common latent space?
- **Additional References:**
  - Uco3D: https://uco3d.github.io/
  - 3DGS: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/
  - 2DGS: https://surfsplatting.github.io/

2DGS, Huang et al. SIGGRAPH 2024

Project Directions

Exact LinPrim Blending

- **Goal:** Improve LinPrim by replacing the blending algorithm
- **Setup:**
  - **Input**: posed RGB images
  - **Output**: LinPrim scene reconstruction
- **Method:**
  - Extend the LinPrim rendering pipeline with exact rendering following EVER
  - Adjust the gradient computation to match the new rendering
- **Possible Extensions:**
  - LinPrim with mesh rasterizer + no ray space approximation
- **Additional References:**
  - LinPrim: https://nicolasvonluetzow.github.io/LinPrim/
  - EVER: https://half-potato.gitlab.io/ever/



EVER, Mai et al., ICCV 2025

Project Directions

Image-Conditioned GaussianGPT

- **Goal:** Extend GaussianGPT from unconditional/completion only to image-conditioned generation

- **Setup:**
  - **Input**: RGB images (+poses?)
  - **Output**: 3D Gaussian scene

- **Method:**
  - Encode images using a vision backbone (e.g. DINO)
  - Explore methods for conditioning the GPT model (cross-attention, prefix, …) and fine-tune

- **Possible Extensions:**
  - Text conditioning
  - Few-view reconstruction

- **Additional References:**
  - GaussianGPT: https://nicolasvonluetzow.github.io/GaussianGPT/
  - SAR3D: https://cyw-3d.github.io/projects/SAR3D/

Project Directions

Local 3DGS Geometry Correction

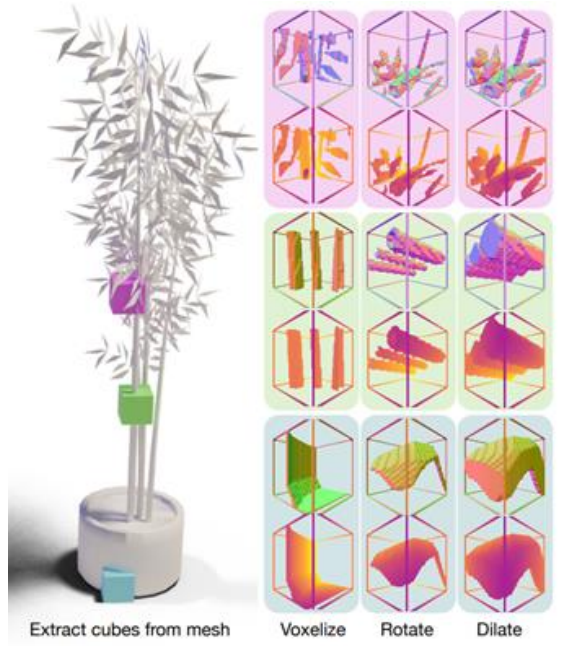
- **Goal:** Lightweight, patch-based correction of geometric artifacts in 3DGS

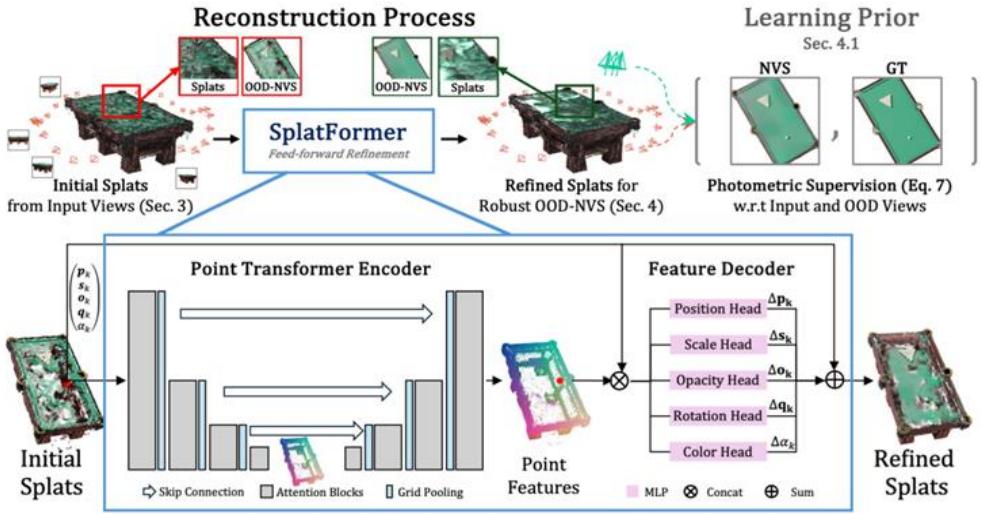
- **Setup:**
  - **Input**: local patch of Gaussians
  - **Output**: per-Gaussian position, scale and rotation residuals

- **Method:**
  - Create data by corrupting patches from high-quality 3DGS scenes
  - Train a small point transformer to undo the corruption

- **Questions:**
  - Does this generalize across scenes and modalities?
  - Can we iteratively refine scenes through multiple applications?
  - Can we refine appearance?

- **Additional References:**
  - SplatFormer: https://sergeyprokudin.github.io/splatformer/
  - 3DGS: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/
  - Nerfbusters: https://ethanweber.me/nerfbusters/





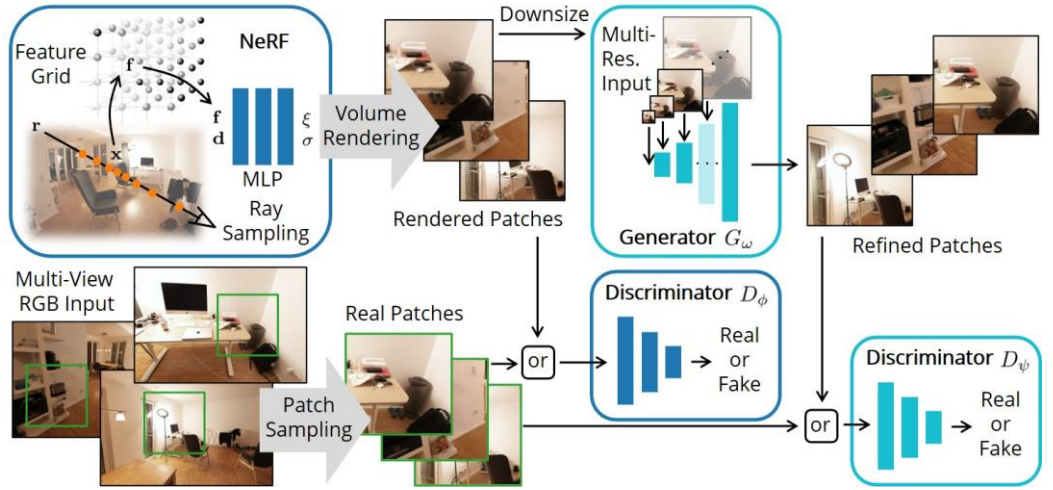
Nerfbusters, Warburg et al., ICCV 2023

SplatFormer, Chen et al., ICLR 2025

Project Directions

GAN-3DGS

- **Goal:** apply the GANeRF paradigm to 3DGS
- **Setup:**
  - **Input+Output:** same as standard 3DGS
- **Method:**
  - Patch-based discriminator classifies real training vs. current rendering
  - Integrate adversarial loss into optimization
  - Likely: introduce stabilization strategies for smooth gradients
- **Questions:**
  - Does this generalize across scenes and modalities?
  - Can we iteratively refine scenes through multiple applications?
  - Can we refine appearance?
- **Additional References:**
  - GANeRF: https://barbararoessle.github.io/ganerf/
  - 3DGS: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/
  - GGHead: https://tobias-kirschstein.github.io/gghead/



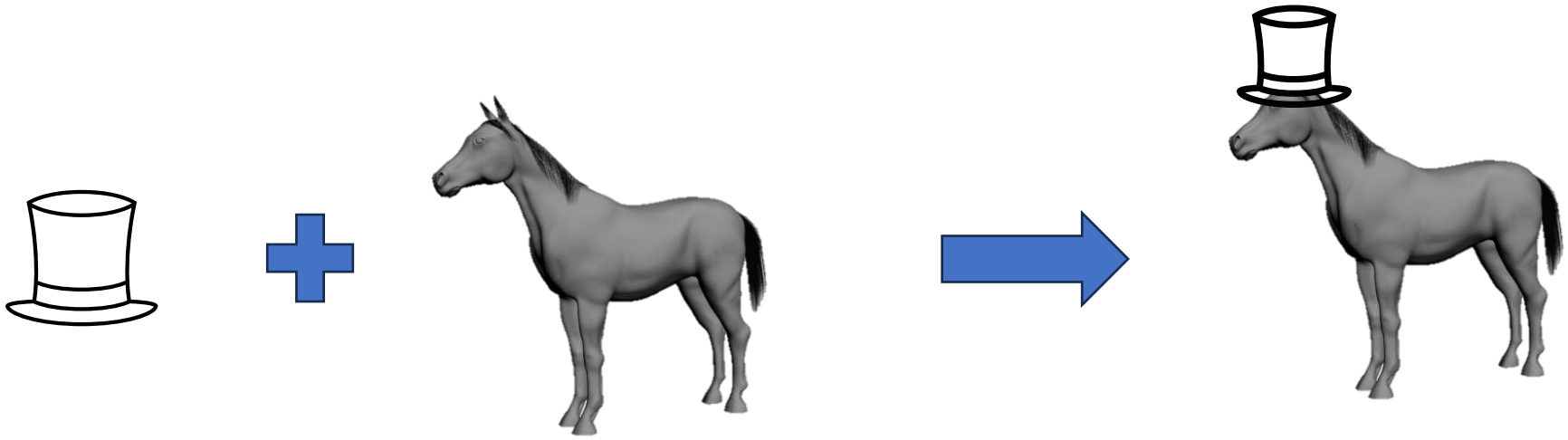
GANeRF, Rössle et al., SIGGRAPH Asia 2023

Generative Modeling and Neural Fields

Ziya Erkoc

Project Directions

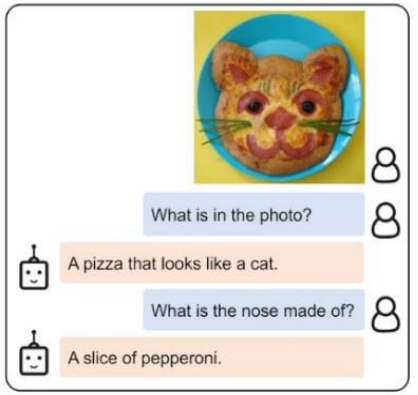
Task 1: Image-based 3D Editing



- **TLDR:** Fast 3D editing with image guidance
- **Input:** Image + 3D shape
- **Output:** Edited 3D Shape
- **Related works:** PrEditor3D (https://arxiv.org/abs/2412.06592), MaskedLRM (https://arxiv.org/abs/2412.08641), ImageDream (https://arxiv.org/abs/2312.02201)
- **Tutor:** Ziya Erkoc (ziya.erkoc@tum.de)

Project Directions

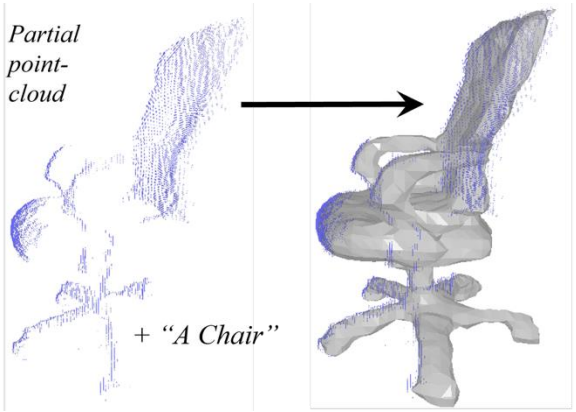
Task 2: Zero-Shot 3D Shape Correspondance with VLMs

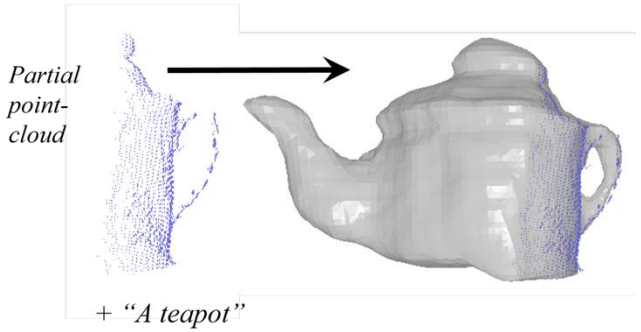


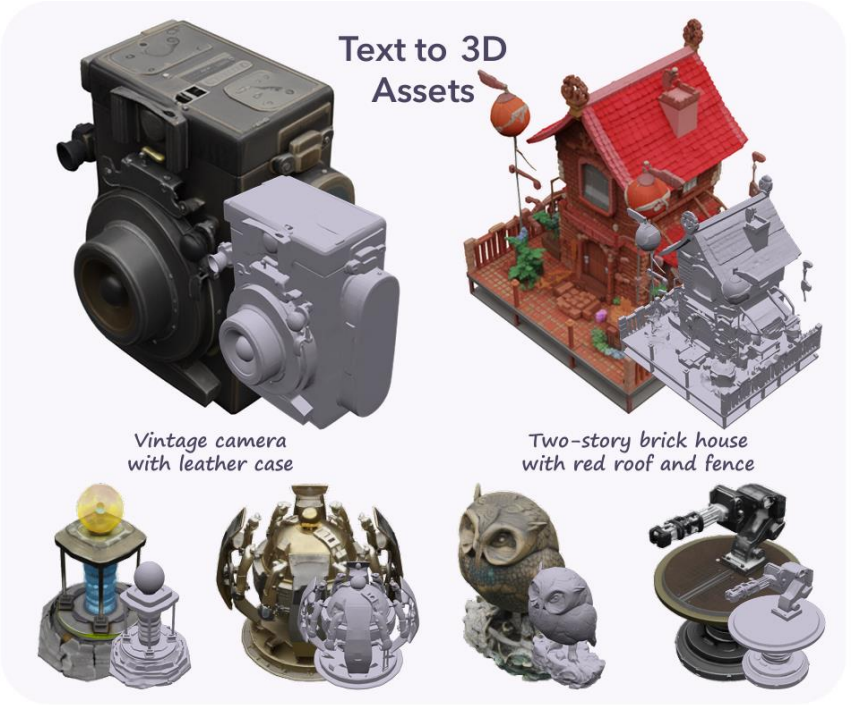
- **TLDR:** Find correspondances between 3D shapes using VLMs
- **Input:** Two input 3D shapes
- **Output:** Point-wise or face-wise or region-wise correspondance map
- **Related works:** Zero-Shot 3D Shape Correspondance (https://arxiv.org/pdf/2306.03253)
- **Tutor:** Ziya Erkoc (ziya.erkoc@tum.de)

Project Directions

Task 3: Zero-Shot 3D Shape Completion







- **TLDR:** Complete partial 3D shapes with knowledge from pre-trained Foundation Models.
- **Input:** A partial 3D shape
- **Output:** A completed 3D shape
- **Related works:** PointCloud Completion with T2I (https://arxiv.org/abs/2306.10533), TRELLIS (https://arxiv.org/abs/2412.01506)
- **Tutor:** Ziya Erkoc (ziya.erkoc@tum.de)

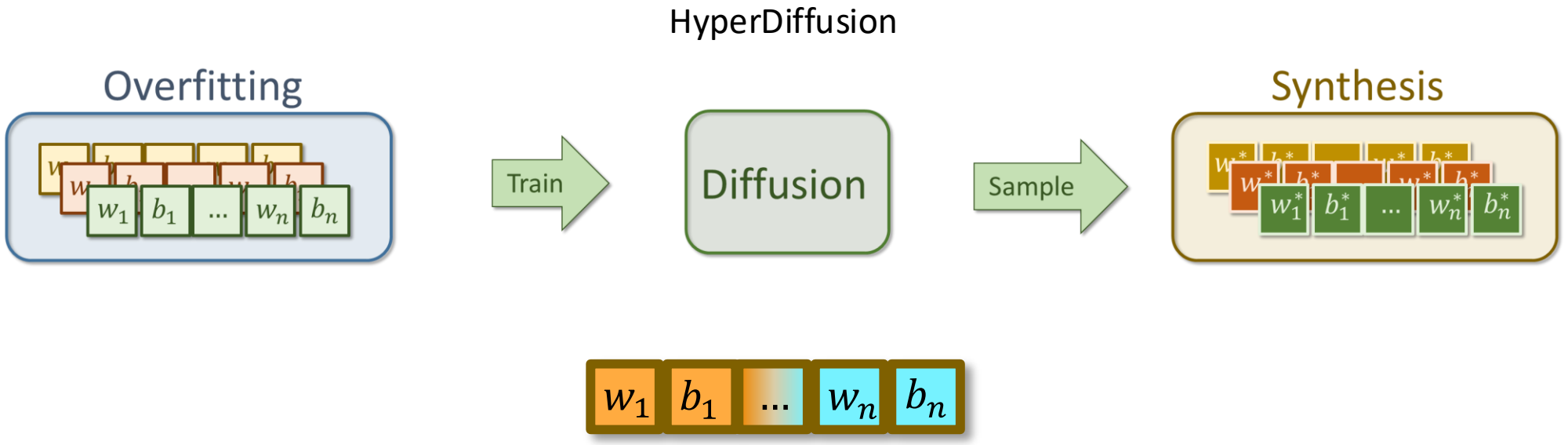
Project Directions

Task 4: LLM on Neural Field Weights

- **TLDR:** Generate MLP weights in an autoregressive fashion
- **Input/Output:** MLP weights of a neural field
- **Related works**: HyperDiffusion (https://arxiv.org/abs/2303.17015), MeshGPT (https://arxiv.org/abs/2311.15475)
- **Tutor:** Ziya Erkoc (ziya.erkoc@tum.de)

Project Directions

Task 5: Semantic-Decomposition for HyperDiffusion



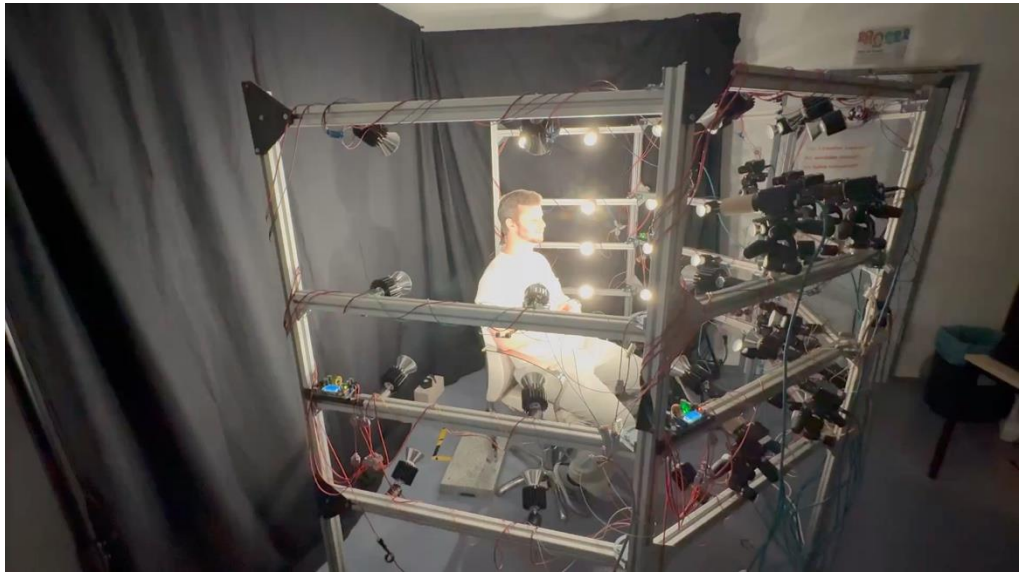
- **TLDR:** Run HyperDiffusion on a semantically decomposed MLP space
- **Input/Output:** Semantically decomposed MLP weights of a Neural Field
- **Related works:** HyperDiffusion (https://arxiv.org/abs/2303.17015)
- **Tutor:** Ziya Erkoc (ziya.erkoc@tum.de)

Head Avatars, Relighting & Inverse Rendering

Jonathan Schmidt
jonathan.schmidt@tum.de

Project Directions

INTRO: Photorealistic Relightable Avatars



Light Stage Scans

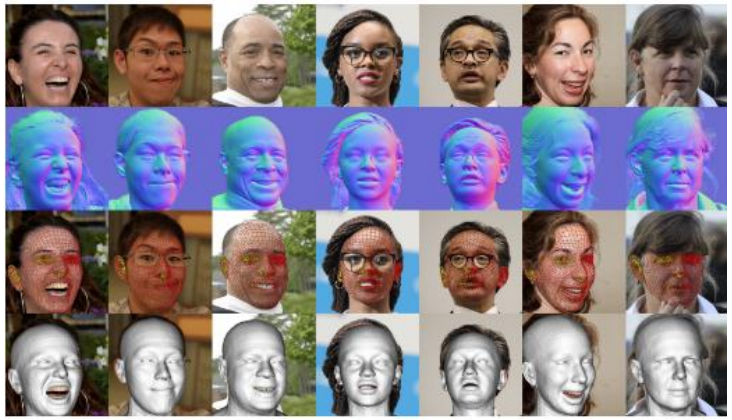
Animation & Relighting

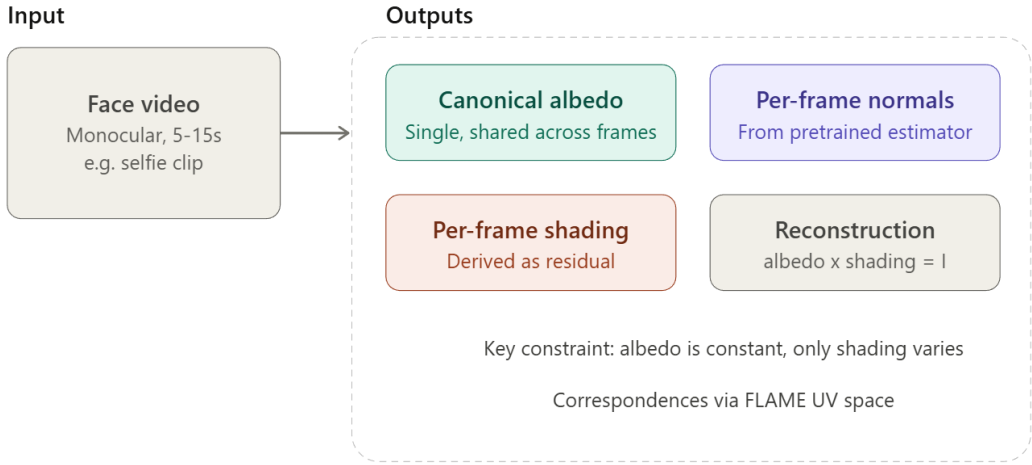
Input: Studio Captures or Casually Captured Videos

Output: Relightable & Animatable Avatar

Project Directions

Project 1: Intrinsic Decomposition from Monocular Face Videos





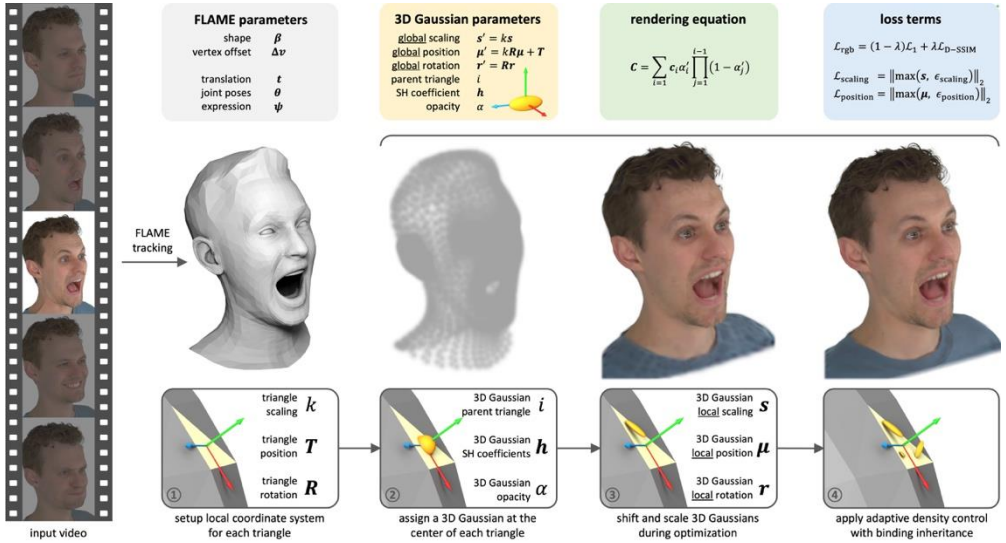
- **Input**: Monocular Video of Face
- **Output**: Canonical albedo, per-frame shading and normal maps
- **Method** Generate per-frame FLAME UV maps and normals using Pixel3DMM. Optimize a single shared albedo in UV space and per-frame shading maps.

Resources:

- Sapiens: https://github.com/facebookresearch/sapiens
- Pixel3DMM: https://simongiebenhain.github.io/pixel3dmm/
- DiffusionRenderer: https://research.nvidia.com/labs/toronto-ai/DiffusionRenderer/
- SPARK: https://kelianb.github.io/SPARK/

Project Directions

Project 2: Perceptual Face Losses for Gaussian Avatar Optimization



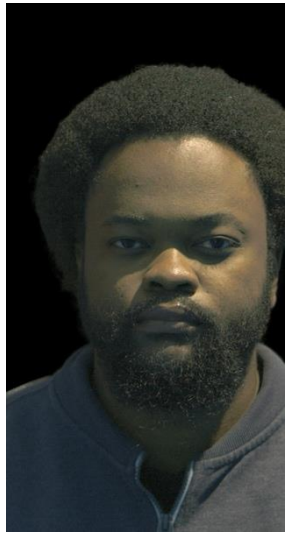
- **Input**: Multi-View Recordings of Faces
- **Output**: High-Fidelity Head Avatar.
- **Method**: Replace generic photometric reconstruction losses with face-specific perceptual objectives - identity, landmarks, expression, and semantic region weighting. Systematically ablate their impact on Gaussian head avatar quality.

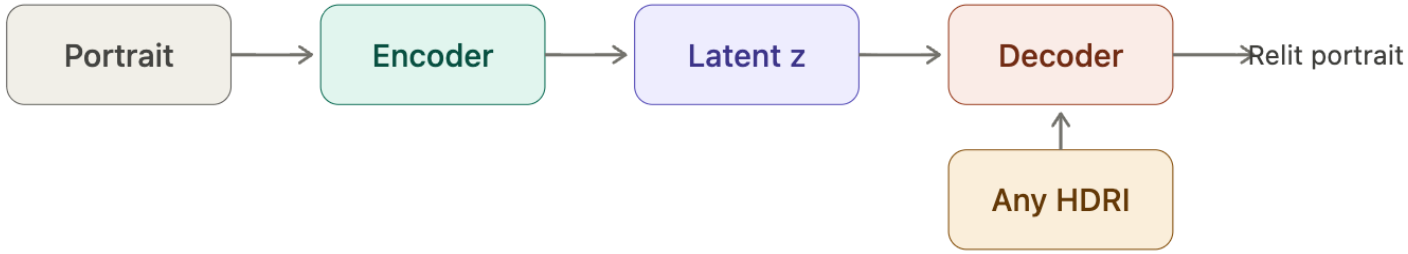
Resources:

- GaussianAvatars : https://shenhanqian.github.io/gaussian-avatars ,
- ArcFace: https://arxiv.org/abs/1801.07698
- OpenGraphAU: https://github.com/lingjivoo/OpenGraphAU

Project Directions

Project 3: Learning a Compact Light Transport Basis for Real-Time Portrait Relighting





- **Input**: Single Portrait Image + HDRI
- **Output**: Relit Image
- **Method**: Train a compact encoder-decoder that compresses the OLAT light transport into a low-dimensional per-pixel code, from which relighted images of the portrait can be decoded

Resources:

- FaceOLAT / 3DPR: https://arxiv.org/abs/2510.15846
- POLAR: https://rex0191.github.io/POLAR/

Project Directions

Project 4: Environment Map Estimation from a Single Face Image

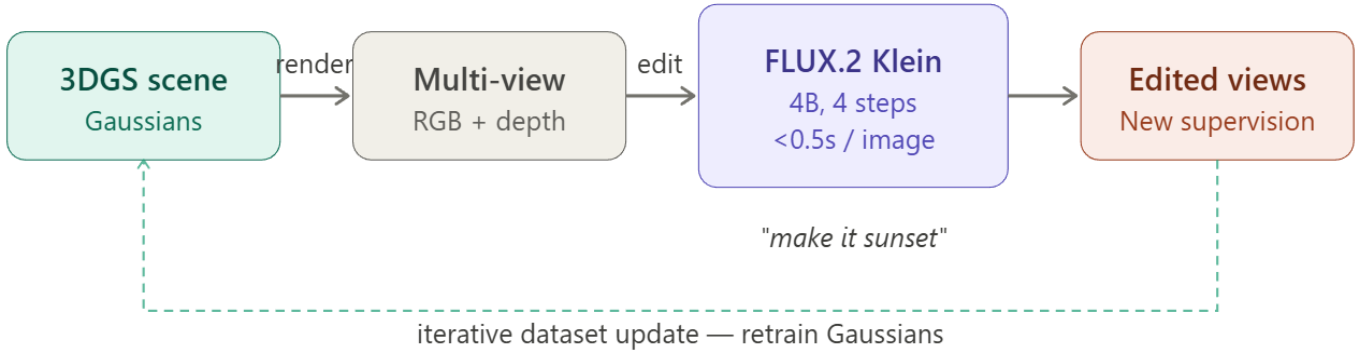
- **Input**: Single or few face images
- **Output**: Estimated Scene Lighting as HDRI
- **Method**: Use image-based relighting to generate a paired dataset between HDRIs and illuminated faces. Then train a NN to map from face images to HDRI.

Resources:

- FaceOLAT / 3DPR: https://arxiv.org/abs/2510.15846
- POLAR: https://rex0191.github.io/POLAR/

Project Directions

Project 5: Text-Guided Appearance Editing of 3D Gaussian Scenes via a Lightweight Diffusion Prior



- **Input**: Reconstructed 3DGS scene + text description of target appearance
- **Output**: Relit 3D Scene
- **Method**: Use a small distilled diffusion model (FLUX.2 Klein) as a 2D prior to edit the appearance of an existing 3DGS scene.
  Implement an iterative SDS-like or dataset-update-style loop where the diffusion model guides per-Gaussian color/opacity updates.

Resources:

- 3DGS: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/
- Flux.2-klein: https://bfl.ai/models/flux-2-klein
- Instruct-GS2GS: https://instruct-gs2gs.github.io/

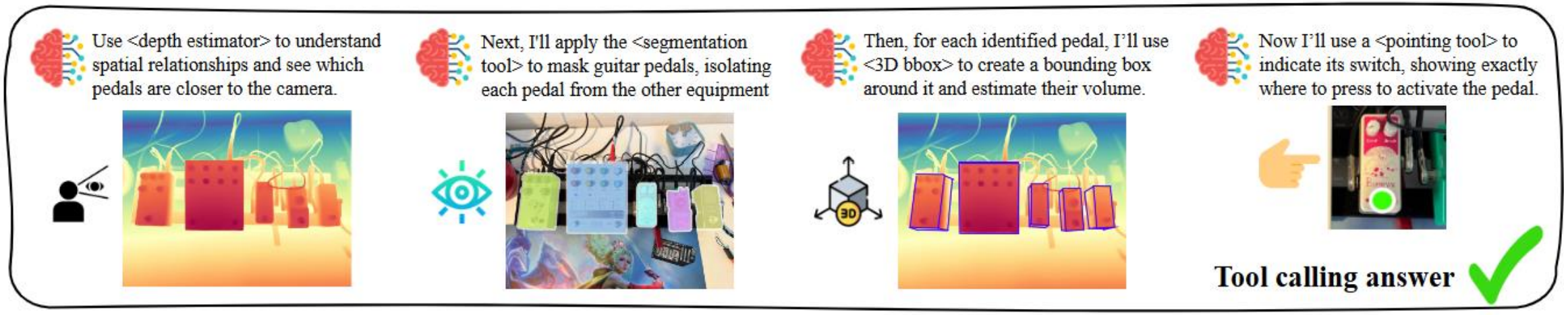
Vision and Language Models

Bartłomiej Baranowski
bartlomiej.baranowski@tum.de

Tool-Augmented Spatial Reasoning

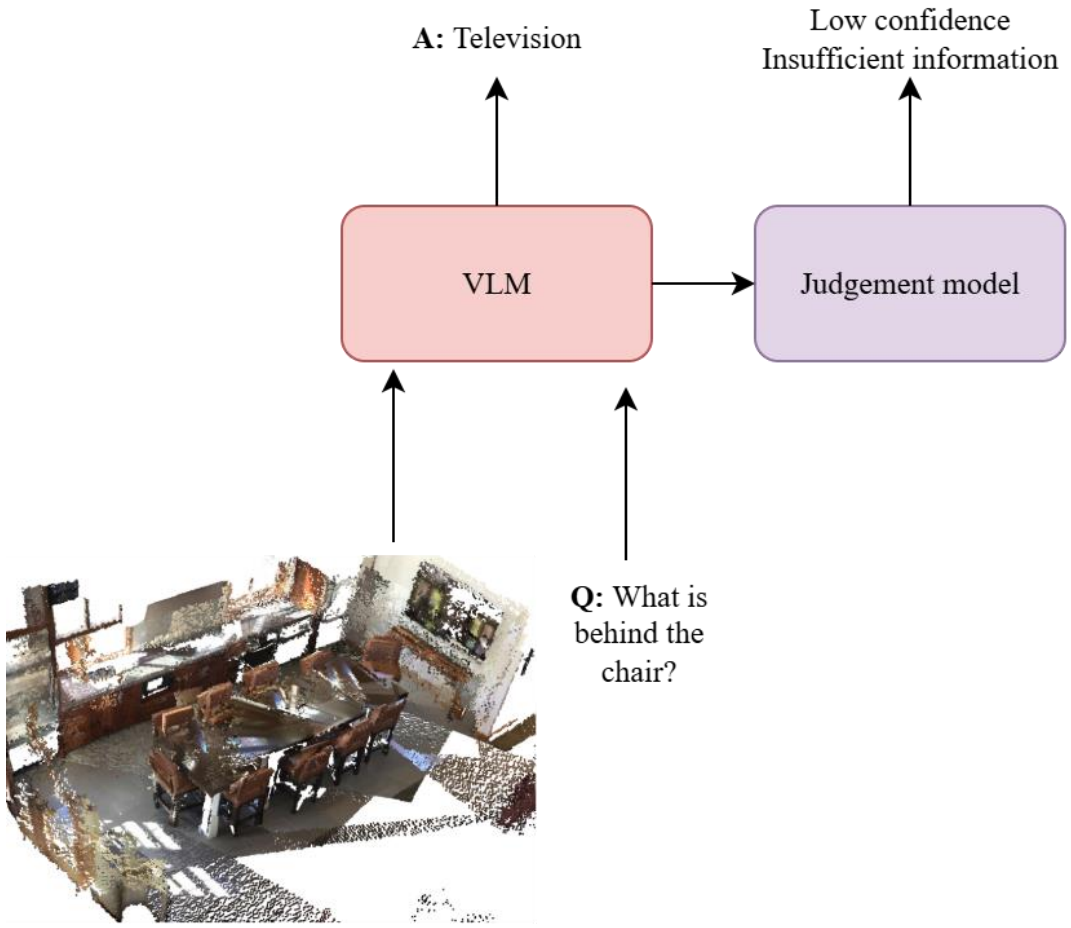
- **Goal:** Teach a VLM to use external tools (e.g. a depth estimator, calculator) for spatial reasoning
  - **Setup:**
    - **Input**: Image(s) of a 3D scene + spatial question
    - **Output**: Answer grounded in tool-extracted measurements
  - **Method:**
    - Define a list of available tools, and the format with which the VLM can access them
    - Fine-tune a small VLM (e.g. Qwen3-VL) with LoRA to generate tool calls

- **Open questions:**
  - Which tools help which question types?
  - Can the model learn when not to use and when not to use a tool
  - Can it be used in a 3D setting?
- **Additional References:**
  - SpaceTools: https://arxiv.org/abs/2512.04069
  - VSI-Bench: https://arxiv.org/abs/2404.09278



Selective Prediction and Failure Attribution for Spatial VLMs

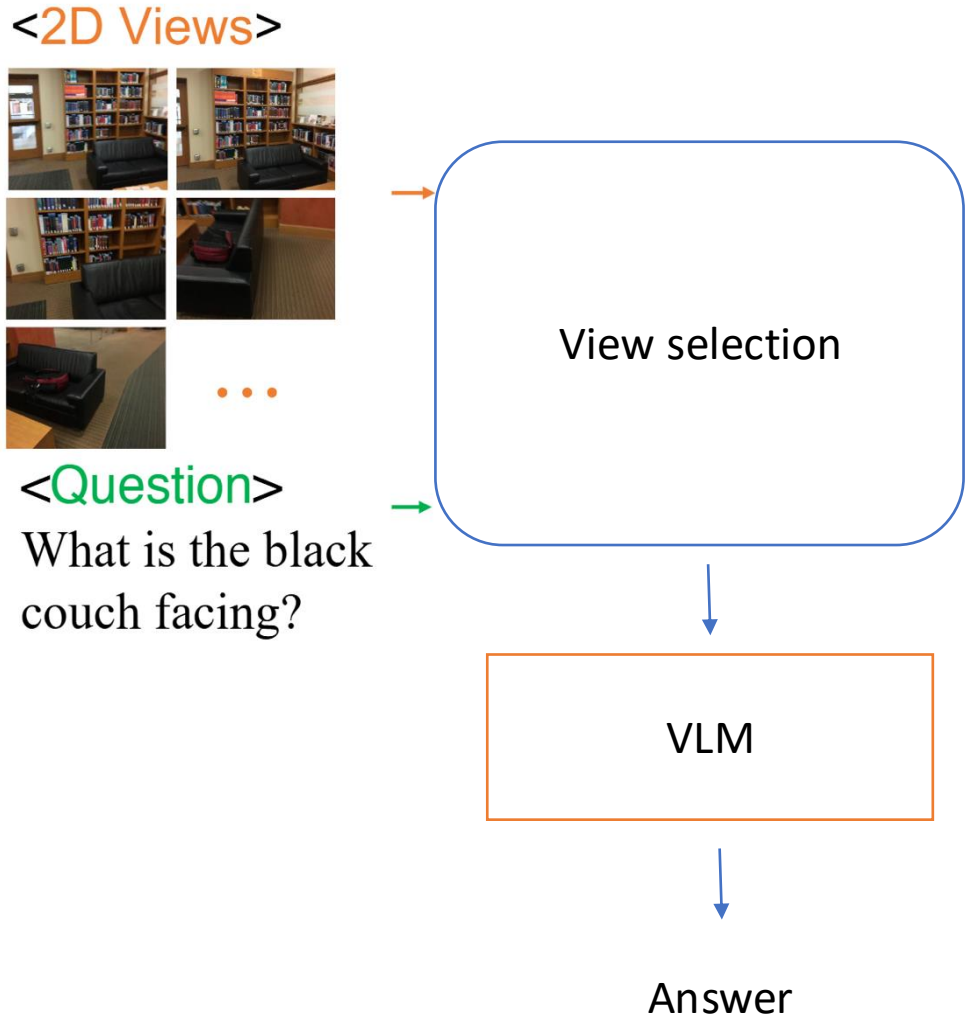
- **Goal:** Build a confidence estimator for 3D VLM spatial answers that predicts correctness, with an auxiliary cause tag where possible, using 3D scene metadata
  - **Setup:**
    - **Input**: VLM internals during spatial QA on 3D scenes + scene metadata
    - **Output:** Per-answer reliability score for selective abstention, with an auxiliary cause tag where possible
  - **Method:**
    - Derive aleatoric signals from geometry: project target bboxes into frames, run depth occlusion test, flag low coverage or missing frames
    - Construct ambiguity probes: strip referring expressions ("the red chair by the window" becomes "the chair") to manufacture referent ambiguity with known labels
    - Train a lightweight classifier from VLM internals predicting correctness, tag flagged failures as aleatoric when geometry or ambiguity probes support it, epistemic otherwise
  - **Open questions:**
    - Do hidden-state probes beat logprob baselines on spatial QA?
    - Can failures be meaningfully decomposed into aleatoric vs epistemic, or do the causes collapse in practice?
    - Can we improve explainability of VLMs over just asking for longer explanations?
  - **Additional References:**
    - EnsemHalDet: https://arxiv.org/abs/2604.02784
    - ICR Probe: https://arxiv.org/abs/2507.16488
    - CAPTURE: https://arxiv.org/abs/2504.15485
    - AHA: https://aha-vlm.github.io/



3D VLM View Selection

- **Goal:** Given N candidate views of a 3D scene, select a subset or just parts of images that maximize downstream VLM QA performance

  - **Setup:**
    - **Input**: Multi-view RGB-D images with camera poses + spatial QA benchmark
    - **Output**: An answer chosen based on view selection policy
  - **Method:**
    - Implement view selection based on its 3D coverage, considering occlusion handling
    - Create a learned selector based on images (e.g. avoiding blurry images)
  - **Open questions:**
    - Does 3D coverage-based selection outperform simple baselines (uniform sampling, farthest camera)?
    - What's the minimum number of tokens we need while still preserving high quality responses?
    - Is more information always better? Can passing less information increase metrics in some cases?
    - Can a learned selector match be trained without requiring depths and poses?
  - **Additional References:**
    - cdViews: https://arxiv.org/abs/2505.22143
    - ❿ LSceneLLM: https://arxiv.org/abs/2412.01292
    - ❿ Video-3D LLM: https://arxiv.org/abs/2412.00493



SAM 3D for Language-Conditioned Affordance Prediction

- **Goal:** Use SAM 3D to reconstruct a full object mesh from a single image, then predict language-conditioned affordance directly in mesh space.
  - **Setup:**
    - **Input**: Single RGB image, verb-conditioned instruction ("pick up", "pour into"), optional depth
    - **Output**: Lightweight head (linear/MLP) on frozen patch features predicting per-pixel affordance maps
  - **Method:**
    - Reconstruct object mesh with SAM 3D from a single image
    - Predict affordance on the mesh by either (a) projecting verb-conditioned MLLM patch features onto mesh faces, (b) a lightweight mesh-space head (MLP on vertex features), or (c) multi-view rendering of the mesh and aggregation of 2D affordance predictions
    - Compare against direct 2D prediction (LOCATE, AffordanceLLM), 3D point cloud prediction (PAVLM, LMAffordance3D)
  - **Open questions:**
    - Does mesh-space prediction give more consistent affordance across viewpoints than per-view 2D prediction?
    - Can foundation-scale reconstruction support fine-grained outputs (contact points, graspable edges) that coarse heatmaps cannot?
  - **Additional references:**
    - SAM 3D: https://ai.meta.com/sam3d/
    - LMAffordance3D (CVPR 2025): https://arxiv.org/abs/2504.04744
    - PAVLM: https://arxiv.org/abs/2410.11564

VLMs as Zero-Shot Reward Models for Visual RL

A robot is doing manipulation tasks in simulation, e.g. pushing, reaching, opening, placing, ... Normally, RL needs a reward like r_t = distance reduction to target. But designing these rewards manually is annoying and task-specific.

Idea: Test whether a VLM can act as a reward function for a robot RL task, without manually designing the reward. Show the VLM frame pairs (before/after) and goal text, the VLM judges whether the robot made progress.

- **Goal:** See if a VLM can judge manipulation task progress from raw pixels, ranking rollouts by success without any reward engineering
  - **Setup:**
    - **Input**: A VLM (e.g. Qwen3-VL), a simulation benchmark (e.g. Meta-World), rollouts at varying success rates
    - **Output**: VLM-based reward scoring pipeline judging progress
  - **Method:**
    - Present VLM with frame pairs (before/after) and goal text, create progress judgments which can be used for training and RL agent
      i.e. converting the VLM output to a numerical reward signal for RL.
      r_t = VLM(image_t, image_{t+1}, goal)
    - Prove it to be working in a real agent case to improve success rate of tasks
      Compare VLM reward with random/hand-designed reward
  - **Open questions:**
    - Can a frozen VLM reliably rank rollouts by progress from pixel pairs alone? i.e. we do not train the VLM, we just prompt it
    - Does the VLM need some fine-tuning to be reliable for training a RL model
    - Where does VLM judgment break down: visually similar states, long-horizon tasks, subtle differences, complex tasks?
  - **Additional References:**
    - Rocamonde et al.: https://arxiv.org/abs/2310.12921
    - Eureka: https://arxiv.org/abs/2412.01292
    - Meta-World: https://meta-world.github.io/

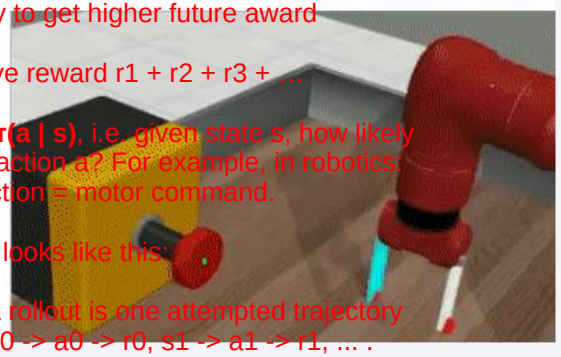
Reinforcement Learning:
- state_t -> agent choose action_t
- action_t -> environment changes
- environment returns state_{t+1}, reward_t
- agent updates policy to get higher future award

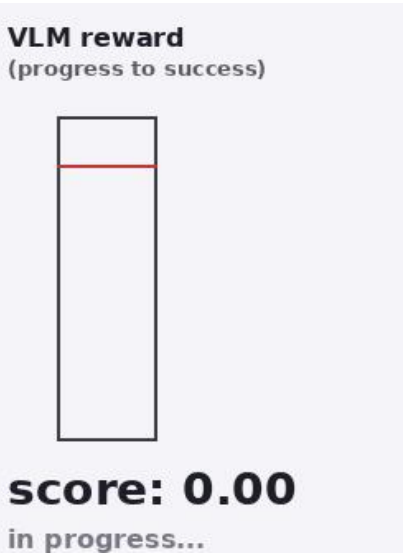
Goal: Maximize cumulative reward r1 + r2 + r3 + ...

A RL model is a **policy: π(a | s)**, i.e. given state s, how likely should the agent choose action a? For example, in robotics: State = camera image, action = motor command.

One training cycle often looks like this:

Step 1: Collect rollouts. A rollout is one attempted trajectory using the current policy: s0 -> a0 -> r0, s1 -> a1 -> r1, ...





Step 2: Each state-action pair (s_t, a_t) is evaluated by estimating how much future reward does a_t lead to, i.e. G_t = r_t + γ r_{t+1} + γ² r_{t+2} + γ³ r_{t+3} + ...

Step 3: Update the policy. If action a_t in state s_t led to higher-than-expected future reward, make this action more likely next time and vice versa.

Ranking rollouts by success:
- Rollout A: Robot successfully presses button.
- Rollout B: Robot moves closer but fails.
- Rollout C: Robot does nothing useful.

The VLM should judge A > B > C, meaning A is best, B is partial progress, C is bad.

From assigning a reward for frame pairs to a full rollout score: A rollout contains many frame pairs (I0, I1), (I1, I2), (I2, I3), ... The VLM scores each pair: r0, r1, ..., rT, then we sum the rewards R = r0 + r1 + ... + rT. Now the whole rollout has a score, and we can rank rollouts.